# Prompts utilizados en ProyectoBanco

*Registro de solicitudes y resultados por fases*

Solicitudes originales, fragmentos y síntesis identificadas

Entrega 30 de octubre de 2026

# Contenido

1 Análisis y planeación ..... 3
2 Creación del backend ..... 3
3 Configuración de PostgreSQL ..... 3
4 Registro y autenticación ..... 3
5 Gestión de cuentas ..... 3
6 Transferencias ..... 4
7 Histórico de movimientos ..... 4
8 Generación de QR ..... 4
9 Pruebas y validaciones ..... 4
10 Refactorización a monolito modular ..... 4
11 Rediseño visual con frontend design ..... 5
12 Organización del frontend por módulos ..... 5
13 Documentación ..... 5
14 Verificación final ..... 5
15 Preparación de la entrega ..... 5
Anexo histórico conservado ..... 6
Prompts y decisiones utilizados ..... 6
Resumen de instrucciones operativas del asistente ..... 6
Solicitud literal de revisión — 24 de octubre de 2026 ..... 7
Solicitud literal de rediseño con frontend-design — 24 de octubre de 2026 ..... 16
Solicitud de navegación modular —24-10-2026 ..... 16
Organización por módulos original completo recuperado ..... 17
Solicitud final de entrega original completo ..... 18

Registro académico de solicitudes reales, fragmentos y síntesis con procedencia. Herramienta: Codex en esta conversación y skills OmniSpec/frontend-design invocadas. No se atribuye trabajo en otros IDE sin evidencia.

Las fases técnicas se agrupan para sustentación; varias se implementaron con una sola invocación omni-implement. No se inventan prompts separados para cada archivo ni se reproduce razonamiento interno.

# 1 Análisis y planeación

Objetivo: Definir alcance y stack.

Prompt o referencia disponible: «Acabo de subir un PDF, revisalo»; «El frontend lo requiero con react y el backend tiene que estar documentado con javadoc»

Procedencia y límite: Fragmentos literales conservados. Invocaciones reales omni-init, omni-discuss, omni-specify y omni-plan.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Especificación 001 y decisiones de stack.

# 2 Creación del backend

Objetivo: Implementar la API.

Prompt o referencia disponible: Invocación omni-implement

Procedencia y límite: No se conserva un prompt independiente completo para backend; la invocación aplica el plan aprobado.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Backend Spring Boot con Javadoc.

# 3 Configuración de PostgreSQL

Objetivo: Persistir y aislar pruebas.

Prompt o referencia disponible: «Delego esas decisiones técnicas y visuales»

Procedencia y límite: Respuesta literal. No existe un prompt SQL independiente disponible.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Esquema y scripts de bases aisladas.

# 4 Registro y autenticación

Objetivo: Acceso por titular.

Prompt o referencia disponible: «Si, necesita inicio de sesion»; «Aprobar propuesta»

Procedencia y límite: Fragmento y respuesta literal; no se presenta el mensaje parcial como original completo.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Registro, sesión y CSRF.

# 5 Gestión de cuentas

Objetivo: Crear apertura única.

Prompt o referencia disponible: «Tu recomendacion me gusta, apliquemosla»; «Me parece perfecto»

Procedencia y límite: Fragmentos literales. Saldo ficticio y cuenta automática son síntesis del alcance.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Una cuenta y una apertura de 1000000 COP.

# 6 Transferencias

Objetivo: Preservar exactitud.

Prompt o referencia disponible: «Hasta dos decimales»

Procedencia y límite: Respuesta literal. No hay un prompt específico completo separado del workflow.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: BigDecimal, transacción y misma clave de reintento.

# 7 Histórico de movimientos

Objetivo: Consultar movimientos propios.

Prompt o referencia disponible: Invocación omni-implement

Procedencia y límite: No hay prompt específico independiente disponible; el enunciado y el plan guían esta fase.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Apertura, enviados y recibidos paginados.

# 8 Generación de QR

Objetivo: Mostrar datos confirmados.

Prompt o referencia disponible: Invocación omni-implement

Procedencia y límite: No hay prompt técnico independiente conservado.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: QR desde comprobante, decodificado por jsQR.

# 9 Pruebas y validaciones

Objetivo: Comprobar regresiones.

Prompt o referencia disponible: Solicitud literal de revisión del anexo histórico

Procedencia y límite: Original completo disponible en el anexo; no se inventa solicitud adicional.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Pruebas Java, Vitest y Playwright.

# 10 Refactorización a monolito modular

Objetivo: Separar responsabilidades.

Prompt o referencia disponible: Solicitud literal de revisión del anexo histórico

Procedencia y límite: Original completo conservado.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Revisión 002 por funcionalidad.

# 11 Rediseño visual con frontend design

Objetivo: Mejorar presentación.

Prompt o referencia disponible: Solicitud literal de rediseño del anexo histórico

Procedencia y límite: Original recibido termina en Asegúra; no se completa artificialmente.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Identidad fintech 003.

# 12 Organización del frontend por módulos

Objetivo: Separar rutas.

Prompt o referencia disponible: «Transformar el frontend actual en una aplicación bancaria profesional con navegación real por módulos, mediante React Router y páginas independientes.»

Procedencia y límite: Fragmento literal; el original completo recuperado de la conversación aparece antes de la solicitud final.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: React Router y páginas 004.

# 13 Documentación

Objetivo: Preparar informe y guía.

Prompt o referencia disponible: Solicitud actual completa al final

Procedencia y límite: Original íntegro del adjunto recibido 29-10-2026.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Documentos de esta entrega.

# 14 Verificación final

Objetivo: Repetir catálogo.

Prompt o referencia disponible: Solicitud actual completa al final

Procedencia y límite: Misma solicitud, apartados 7 y 11; no es otro prompt histórico.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Nueve checks y 50 casos registrados.

# 15 Preparación de la entrega

Objetivo: Organizar archivos y Git.

Prompt o referencia disponible: Solicitud actual completa al final

Procedencia y límite: Misma solicitud, apartados 10 a 13.

Herramienta: Codex y workflow del proyecto. Resultado contrastado con archivos: Entrega local en developer; sin push.

## Anexo histórico conservado

Se conserva el registro anterior con sus originales y síntesis etiquetadas. La tabla inicial contiene respuestas breves, no transcripciones completas.

## Prompts y decisiones utilizados

Registro del trabajo asistido en esta conversación. Las citas siguientes son solicitudes reales del usuario; las decisiones indicadas como resumen no son prompts inventados ni instrucciones atribuidas al profesor.

| Etapa | Prompt o respuesta real | Resultado |
|---|---|---|
| Onboarding | Invocación de omni-init | Inventario y contexto del proyecto |
| Fuente académica | «Acabo de subir un PDF, revisalo» | Lectura del ejercicio de la página 11 y detección de contradicción Angular/React |
| Discusión | «El frontend lo requiero con react y el backend tiene que estar documentado con javadoc» | React y Javadoc confirmados |
| Acceso | «Si, necesita inicio de sesion» y consulta sobre alcance | Propuesta de registro/cuenta automática |
| Alcance | «Tu recomendacion me gusta, apliquemosla»; confirmó saldo ficticio, COP y cuenta automática | Simulación acordada; registro y transferencias entre cuentas |
| Saldo | «Me parece perfecto» a $1.000.000 COP inicial y reglas propuestas | Decisión de alcance registrada |
| Requisitos | Invocación de omni-specify | Criterios estables y exclusiones |
| Diseño | Invocación de omni-plan | API, persistencia, sesiones, reintentos, QR, UI y verificación |
| Aclaraciones de diseño | «Aprobar propuesta», «Hasta dos decimales», «Delego esas decisiones técnicas y visuales» | Nombre/correo/contraseña; 30 minutos de sesión; precisión; herramientas y UI delegadas |
| Tareas | Invocación de omni-tasks | Once tareas trazadas a criterios |
| Implementación | Invocación de omni-implement | Código de backend/frontend, scripts, pruebas y documentación |

## Resumen de instrucciones operativas del asistente

Se aplicaron las skills del proyecto invocadas por el usuario. El asistente inspeccionó el repositorio, consultó fuentes oficiales registradas en el plan, escribió el código y ejecutó comandos de compilación/documentación/pruebas. Las instrucciones de implementación se derivaron del alcance aprobado: validar titularidad, exactitud monetaria, atomicidad, concurrencia, identidad de reintento, histórico

privado y QR desde comprobante confirmado. Este párrafo resume el proceso, no reproduce un prompt literal de generación separado.

Las herramientas ejecutadas y sus resultados están en verification.md. No se incluyen contraseñas reales, tokens, contenido de configuración local ni prompts de sistema. Los únicos datos de acceso documentados en README son usuarios ficticios públicos de demostración.

# Solicitud literal de revisión — 24 de octubre de 2026

```
Quiero que realices una revisión completa y una mejora controlada del proyecto existente
`ProyectoBanco`.

IMPORTANTE: este proyecto YA FUNCIONA. No debes reconstruirlo desde cero ni reemplazar
funcionalidades correctas innecesariamente. Primero inspecciona completamente el backend, frontend,
base de datos, pruebas, scripts y documentación existente. Después realiza una refactorización
incremental conservando el comportamiento actual.

OBJETIVO ACADÉMICO

El proyecto debe cumplir completamente con el siguiente ejercicio:

“Generar una aplicación completa para hacer el traslado de dinero entre cuentas en un banco. Debe
mostrar el histórico de transacciones y, cuando se genere el resultado de la transacción, mostrar un
código QR con los datos básicos de la transacción codificados. Se debe entregar informe del
ejercicio, código en GitHub y prompts utilizados.”

Fecha de entrega: 30 de octubre de 2026.

Actualmente el proyecto ya implementa transferencias, histórico, QR, registro, login, PostgreSQL,
seguridad, validaciones, concurrencia, idempotencia, pruebas y documentación. Debes conservar y
mejorar estas funcionalidades.

REGLAS IMPORTANTES

- No elimines funcionalidades existentes que estén funcionando.
- No simplifiques reglas de seguridad o integridad existentes.
- No reemplaces BigDecimal por double o float.
- No elimines la idempotencia de transferencias.
- No elimines los bloqueos/transacciones utilizados para proteger los saldos.
- No cambies PostgreSQL por H2 para las pruebas de integración que requieren comportamiento real.
- No elimines CSRF, autenticación, autorización ni manejo de sesiones.
- No cambies React por Angular.
- No conviertas el proyecto en microservicios.
- No agregues complejidad sin una razón concreta.
- No modifiques contratos HTTP innecesariamente.
- Si debes modificar un endpoint o DTO, actualiza frontend y pruebas correspondientes.
- Todos los nombres de clases, métodos, variables, atributos y paquetes Java deben permanecer en
inglés.
- Los DTO del backend deben implementarse con Java records cuando corresponda.
- Todo método público no trivial del backend debe tener Javadoc claro y actualizado.
- No deben existir secretos, contraseñas reales o datos locales dentro del repositorio Git.
- Antes y después de cambios importantes ejecuta las pruebas correspondientes.

ARQUITECTURA OBJETIVO DEL BACKEND

Quiero mantener una sola aplicación Spring Boot y una sola base de datos PostgreSQL, utilizando un
MONOLITO MODULAR organizado principalmente por funcionalidad.
```

Reorganiza progresivamente el paquete `co.edu.tdea.bank` aproximadamente de la siguiente manera:

```
co.edu.tdea.bank
├── BankApplication.java
├── auth
│   ├── controller
│   ├── dto
│   ├── service
│   ├── entity
│   └── repository
├── account
│   ├── controller
│   ├── dto
│   ├── service
│   ├── entity
│   └── repository
├── transfer
│   ├── controller
│   ├── dto
│   ├── service
│   ├── entity
│   └── repository
├── movement
│   ├── controller
│   ├── dto
│   ├── service
│   └── repository
└── shared
    ├── config
    ├── security
    └── exception
```

Esta estructura es una referencia. Puedes realizar pequeños ajustes si existe una razón arquitectónica clara, pero conserva el principio de monolito modular por funcionalidad.

Refactoriza el actual `BankController` para evitar que un solo controlador maneje autenticación, cuentas, movimientos y transferencias.

Preferiblemente deben existir responsabilidades similares a:

- AuthController
- AccountController
- TransferController
- MovementController

Refactoriza el actual `BankService` para evitar que concentre registro, usuarios, cuentas, histórico y transferencias en una única clase.

Utiliza servicios cohesivos, por ejemplo:

- AuthService
- AccountService
- TransferService
- MovementService

No dividas servicios artificialmente. Cada servicio debe agrupar operaciones relacionadas.

La transacción monetaria completa debe permanecer dentro de un único límite transaccional y continuar garantizando:

- débito del origen;
- crédito del destino;
- registro de la transferencia;
- rollback completo ante cualquier error;
- protección frente a concurrencia;
- saldo nunca negativo;
- idempotencia ante reintentos.

SEPARACIÓN DE ARCHIVOS

Actualmente existen archivos que contienen varios tipos, como `Contracts.java` y `Repositories.java`.

Organízalos en archivos separados cuando mejore la claridad.

Ejemplos:

- RegistrationRequest
- LoginRequest
- IdentityResponse
- AccountResponse
- TransferRequest
- TransferReceiptResponse
- MovementResponse
- MovementPageResponse
- ErrorResponse

Mantén los DTO apropiados como Java records.

Igualmente, separa:

- UserRepository
- AccountRepository
- TransferRepository

en sus archivos y módulos correspondientes.

PERSISTENCIA

Conserva PostgreSQL, Spring Data JPA y el esquema existente.

Revisa el uso directo de `JdbcTemplate` dentro de servicios.

Cuando exista SQL específico para consultas de histórico o read models, aísla esas consultas en una clase de repositorio/query repository apropiada en lugar de dejar SQL dentro del servicio de aplicación.

No cambies una consulta correcta simplemente por utilizar JPA si hacerlo empeora la implementación.

Mantén:

- BigDecimal en Java;
- NUMERIC(19,2) en PostgreSQL;
- restricciones de integridad;
- claves únicas;
- idempotency key;

- índices necesarios;
- bloqueo seguro de cuentas;
- actualización atómica.

FRONTEND

Conserva React + TypeScript + Vite.

El frontend actual funciona y visualmente tiene una buena base. No quiero reemplazarlo por un diseño completamente diferente.

Quiero mejorar tanto la arquitectura del código como la experiencia visual.

Organiza `src` aproximadamente así:

```
src
├── app
│   └── App.tsx
├── features
│   ├── auth
│   ├── account
│   ├── transfer
│   ├── history
│   └── receipt
├── shared
│   ├── api
│   ├── components
│   ├── types
│   └── utils
└── styles
```

Divide los componentes actuales cuando tengan demasiadas responsabilidades.

Especialmente revisa:

- App.tsx
- AccountPanel.tsx
- api.ts
- domain.ts
- styles.css

No quiero componentes enormes ni código comprimido en líneas excesivamente largas.

Formatea correctamente todos los archivos TypeScript, TSX y CSS.

Puedes configurar Prettier si resulta conveniente y compatible con el proyecto.

Separa componentes visuales reutilizables cuando sea útil, pero no generes componentes artificiales de pocas líneas sin valor.

La interfaz final debe mostrar claramente:

- login;
- registro;
- usuario autenticado;
- cuenta propia;
- número de cuenta;

- saldo;
- moneda COP;
- formulario de transferencia;
- revisión previa de transferencia;
- estado de procesamiento;
- errores de validación;
- transferencia confirmada;
- datos del comprobante;
- código QR;
- histórico;
- transferencias enviadas;
- transferencias recibidas;
- saldo inicial;
- cierre de sesión;
- aviso visible de que se trata de una simulación académica.

MEJORAS DE EXPERIENCIA

Conserva la identidad visual actual y mejórala de manera sutil.

Revisa:

- responsive desktop/tablet/móvil;
- espaciados;
- estados disabled;
- estados loading;
- mensajes de error;
- mensajes de éxito;
- foco de teclado;
- labels accesibles;
- contraste;
- histórico vacío;
- referencia de transferencia;
- presentación del comprobante;
- presentación del QR.

Puedes incorporar acciones como copiar la referencia del comprobante o guardar el QR si pueden implementarse limpiamente, pero no son obligatorias y nunca deben interferir con la transferencia.

Generar, regenerar, copiar o guardar el QR NO debe volver a ejecutar una transferencia.

QR

El QR debe generarse únicamente después de que el backend confirme la transacción.

Debe contener al menos:

- referencia de la transferencia;
- fecha;
- monto;
- moneda;
- cuenta origen parcialmente ocultada;
- cuenta destino parcialmente ocultada.

No debe contener:

- contraseña;
- hash;

- cookie;
- CSRF token;
- datos de sesión;
- secretos;
- información sensible innecesaria.

Comprueba mediante una prueba que el QR generado realmente puede decodificarse y que el contenido coincide con la transferencia confirmada.

HISTÓRICO

Verifica que el histórico sea persistente y pertenezca únicamente al usuario autenticado.

Debe distinguir claramente:

- OPENING;
- SENT;
- RECEIVED.

Debe mostrar, cuando corresponda:

- fecha;
- monto;
- moneda;
- contraparte parcialmente ocultada;
- referencia.

SEGURIDAD

Conserva y revisa:

- Spring Security;
- passwords almacenadas mediante hash seguro;
- sesión de servidor;
- cookie HttpOnly;
- SameSite;
- CSRF;
- logout;
- expiración por inactividad;
- autorización por usuario;
- protección para que un usuario no pueda consultar cuenta, histórico o transferencias privadas de otro.

No confíes en identificadores enviados por el frontend para determinar la cuenta origen. La cuenta origen debe continuar obteniéndose de la identidad autenticada.

PRUEBAS

Esta parte es prioritaria.

Antes de considerar terminada la refactorización debes verificar que existe cobertura real sobre todos los requerimientos del ejercicio.

Conserva las pruebas actuales y agrega únicamente las que falten.

Backend:

- registro;
- correo duplicado;
- login correcto;
- login incorrecto;
- usuario no autenticado;
- consulta de cuenta propia;
- transferencia correcta;
- monto cero;
- monto negativo;
- monto con más de dos decimales;
- saldo insuficiente;
- cuenta destino inexistente;
- misma cuenta;
- transferencia por saldo completo;
- actualización exacta de ambos saldos;
- rollback si falla parte de la operación;
- transferencias concurrentes;
- idempotencia;
- reutilización incorrecta de una idempotency key;
- histórico enviado;
- histórico recibido;
- privacidad del histórico;
- persistencia;
- expiración de sesión.

Frontend/unitarias:

- validación de montos;
- formato monetario;
- generación del payload QR;
- comportamiento del formulario;
- manejo de errores relevantes;
- operación pendiente/reintento.

E2E con Playwright:

Realiza como mínimo un recorrido completo:

Usuario A inicia sesión
→ consulta saldo
→ transfiere dinero al usuario B
→ backend confirma
→ aparece comprobante
→ aparece QR
→ el QR se decodifica correctamente
→ el histórico de A muestra SENT
→ usuario A cierra sesión
→ usuario B inicia sesión
→ el histórico de B muestra RECEIVED
→ los saldos de ambas cuentas son correctos.

También verifica que recargar la aplicación no elimine las transacciones.

VERIFICACIÓN FINAL

Al terminar ejecuta, utilizando los comandos compatibles con el sistema operativo del proyecto:

Backend:

- tests;
- build/package;
- generación de Javadoc.

Frontend:
- unit tests;
- lint;
- build.

Integración:
- pruebas con PostgreSQL real;
- Playwright E2E;
- decodificación real del QR.

No declares que algo funciona únicamente porque compila.

No elimines una prueba para conseguir un build verde.

Si una prueba falla, corrige la causa.

DOCUMENTACIÓN

Actualiza README.md para que una persona que clone el repositorio pueda ejecutar el proyecto sin conocimiento previo.

Debe incluir:

- descripción;
- objetivo académico;
- funcionalidades;
- arquitectura;
- estructura de carpetas;
- stack;
- requisitos;
- instalación;
- configuración PostgreSQL;
- variables de entorno;
- cómo ejecutar backend;
- cómo ejecutar frontend;
- cómo ejecutar pruebas;
- endpoints principales;
- usuarios demo si corresponde;
- explicación del QR;
- advertencia de que todo el dinero es ficticio;
- limitaciones;
- enlace al informe;
- enlace a prompts utilizados.

INFORME

Revisa y mejora `docs/informe.md`.

Debe quedar preparado para convertirlo posteriormente a Word/PDF y contener:

1. Portada.
2. Introducción.
3. Objetivo general.

4. Objetivos específicos.
5. Descripción del problema.
6. Requerimientos.
7. Tecnologías utilizadas.
8. Arquitectura.
9. Estructura del proyecto.
10. Modelo de datos.
11. Funcionamiento de registro/login.
12. Funcionamiento de cuentas.
13. Flujo de una transferencia.
14. Manejo transaccional.
15. Histórico.
16. Generación y contenido del QR.
17. Endpoints.
18. Validaciones.
19. Seguridad.
20. Pruebas realizadas.
21. Resultados de pruebas.
22. Capturas principales.
23. Conclusiones.
24. URL del repositorio GitHub cuando se publique.

PROMPTS

Actualiza `docs/prompts.md`.

Conserva los prompts reales utilizados y agrega este prompt como parte del historial de desarrollo.

No inventes prompts que nunca se utilizaron.

GITHUB Y LIMPIEZA

Prepara el proyecto para GitHub.

Comprueba `.gitignore` y asegúrate de NO publicar:

- `.local/`;
- bases de datos locales;
- contraseñas;
- `.env`;
- `node_modules/`;
- `target/`;
- `dist/`;
- `test-results/`;
- `playwright-report/`;
- logs;
- archivos temporales;
- credenciales.

Revisa `git status` antes de cualquier commit.

No realices push a un repositorio remoto sin que yo lo autorice expresamente.

RESULTADO QUE ESPERO

Al finalizar quiero recibir un resumen con:

```
- arquitectura final;
- árbol principal de carpetas;
- archivos creados;
- archivos movidos;
- archivos modificados;
- decisiones arquitectónicas;
- funcionalidades preservadas;
- mejoras del frontend;
- pruebas agregadas;
- número total de pruebas ejecutadas;
- resultado de cada suite;
- requisitos del ejercicio cubiertos;
- posibles pendientes;
- instrucciones exactas para ejecutar el proyecto.

Prioridad máxima:

1. No romper lo existente.
2. Cumplir el ejercicio del profesor al 100%.
3. Mejorar organización y mantenibilidad.
4. Tener una interfaz limpia y profesional.
5. Tener pruebas que demuestren el funcionamiento.
6. Dejar README, informe y prompts listos para la entrega.
7. Dejar el repositorio limpio y preparado para GitHub.

Trabaja incrementalmente. Después de cada refactorización estructural importante, compila y ejecuta las pruebas relevantes antes de continuar.
```

# Solicitud literal de rediseño con frontend-design — 24 de octubre de 2026

```
Utiliza la skill `frontend-design` para realizar un rediseño visual profesional de ProyectoBanco.
Quiero transformar la aplicación actual en una plataforma bancaria digital moderna, elegante y visualmente atractiva.
Rediseña de manera coherente el login, registro, dashboard, tarjeta de saldo, formulario de transferencia, histórico y comprobante QR.
Toma como referencia la calidad visual de aplicaciones fintech modernas, con una composición cuidada, excelente tipografía, iconografía consistente, colores elegantes, componentes reutilizables y animaciones sutiles.
Puedes realizar cambios importantes en la apariencia y distribución de los elementos. No quiero únicamente modificar colores y márgenes: quiero una mejora visual evidente respecto al diseño actual.
Mantén React, TypeScript y la arquitectura modular existente.
**Es obligatorio conservar intactas las funcionalidades del proyecto.** No modifiques el backend, las API, la base de datos, la seguridad ni la lógica de transferencias.
Asegúra
```

El texto recibido termina en «Asegúra». No se atribuyen instrucciones adicionales al usuario.

# Solicitud de navegación modular —24-10-2026

Fragmento literal del prompt del usuario: «Transformar el frontend actual en una aplicación bancaria profesional con navegación real por módulos, mediante React Router y páginas independientes.»

Requisitos adicionales del mismo mensaje (síntesis, no transcripción): conservar azul profundo/violeta/blanco y arquitectura features; Resumen acotado, Transferencias independiente, Movimientos paginados con filtros compatibles, Comprobantes recuperables sin volver a transferir y perfil sólo con datos reales. Layouts, rutas protegidas, estado de sesión/pending/recarga, escritorio y móvil; no cambiar dinero/SQL/seguridad/HTTP

innecesariamente. Inspeccionar antes de cambiar; ejecutar build/lint/unit/E2E, documentar. Skill frontend-design solicitada explícitamente.

Acciones del agente: inspección de componentes/endpoints y definición previa de reutilización/rutas; React Router exacto con HashRouter documentado; implementación incremental y regresiones sin agentes ni publicación.

# Organización por módulos original completo recuperado

Fuente: mensaje del usuario disponible en esta conversación, 24 de octubre de 2026. La síntesis anterior se conserva como registro histórico, no como única fuente disponible.

```
Quiero continuar mejorando el frontend de ProyectoBanco utilizando la skill `frontend-design` que ya tenemos instalada.

El rediseño visual actual ha mejorado mucho y quiero conservar esa identidad: azul profundo, violeta, blanco, tipografía moderna, iconografía y tarjetas elegantes.

Sin embargo, hay un problema importante: actualmente el dashboard muestra demasiada información concentrada en una misma pantalla y las opciones del menú lateral no se sienten como módulos independientes.

**OBJETIVO PRINCIPAL**

Transformar el frontend actual en una aplicación bancaria profesional con navegación real por módulos, mediante React Router y páginas independientes.

**MÓDULOS**

1. **Resumen:** pantalla inicial con bienvenida, saldo, número de cuenta, actividad reciente y accesos rápidos. No debe contener el formulario completo de transferencia ni todo el histórico.
2. **Transferencias:** página independiente con formulario y flujo visual de datos → revisión → confirmación → comprobante QR. Conservar las validaciones, la idempotencia y la lógica actual.
3. **Movimientos:** página exclusiva para consultar el histórico. Mostrar enviados, recibidos y apertura, con paginación, filtros y búsqueda cuando sean compatibles con el backend.
4. **Comprobantes:** página independiente para consultar transferencias realizadas y sus comprobantes. Verificar qué información permite recuperar el backend actual. No inventar persistencia ni funcionalidades. Si se requiere un endpoint adicional, analizarlo y documentar la necesidad antes de implementarlo.
5. **Mi perfil:** opcional, solamente si puede implementarse con los datos reales existentes.

**DISEÑO**

Conservar el nuevo diseño visual que ya se implementó, pero mejorar su composición y profesionalismo.

Mantener un sidebar persistente en escritorio, navegación clara y una alternativa responsive funcional en dispositivos móviles.

Agregar estados activos para cada ruta, títulos contextuales, transiciones sutiles y jerarquía visual consistente.

Cada página debe sentirse como un módulo completo, no como una sección a la que simplemente se hace scroll.

No construir pantallas de relleno ni funcionalidades falsas.

**ARQUITECTURA**
```

Utilizar React Router para separar rutas.

Crear layouts compartidos para autenticación y aplicación bancaria.

Mantener la organización modular existente en frontend/src/features.

Reutilizar componentes actuales cuando sea posible. No reconstruir el proyecto desde cero.

Conservar sesión, autenticación, protección de rutas, estado de la cuenta y comportamiento de recarga.

**RESTRICCIONES**

- No modificar la lógica monetaria.
- No alterar el esquema PostgreSQL.
- No cambiar contratos HTTP existentes innecesariamente.
- No eliminar validaciones ni seguridad.
- No romper el histórico, las transferencias o el QR.
- No volver a transferir dinero al visitar un comprobante.
- No sustituir el diseño actual por una plantilla genérica.

**PRUEBAS**

Actualizar y ejecutar las pruebas correspondientes a las nuevas rutas y componentes.

Comprobar navegación, autenticación, protección de rutas, transferencias, histórico, QR, recarga y cierre de sesión.

Verificar responsive en escritorio y móvil.

Ejecutar build, lint, pruebas unitarias y E2E. Mantener las pruebas previas aprobadas y actualizar las que dependan legítimamente de la navegación anterior.

Actualizar documentación y prompts utilizados.

**RESULTADO ESPERADO**

Quiero una aplicación bancaria profesional con módulos claramente separados, navegación funcional, arquitectura limpia, diseño visual consistente y todas las funcionalidades anteriores conservadas.

Antes de realizar cambios, inspecciona la estructura existente y define brevemente qué componentes reutilizarás, qué rutas crearás y cuáles serán los cambios necesarios.

Después implementa el trabajo de forma incremental y verifica los resultados.

# Solicitud final de entrega original completo

Fuente: Texto pegado.txt adjunto el 29 de octubre de 2026. Herramienta Codex. Resultado: esta entrega documental sin cambios bancarios ni publicación remota.

# ENTREGA FINAL ACADÉMICA — PROYECTOBANCO

Necesito que prepares de principio a fin la entrega académica definitiva de ProyectoBanco, utilizando todo el contexto que ya tienes sobre el desarrollo, las refactorizaciones, las pruebas realizadas y la arquitectura actual.

**No quiero que vuelvas a desarrollar la aplicación ni que reconstruyas funcionalidades existentes.** El proyecto ya funciona. En esta etapa debemos concentrarnos en la documentación profesional, la verificación final, la preparación del repositorio y la sustentación.

## 1. REQUISITOS DEL PROFESOR

Fecha de entrega: 30 de octubre de 2026.

El profesor establece:

"Siguiendo los pasos realizados, genere una aplicación completa para hacer el traslado de dinero entre cuentas en un banco. Debe mostrar el histórico de transacciones y mostrar además cuando genere el resultado de la transacción un código QR con los datos básicos de la transacción codificados.

Recuerde que puede usar el IDE Cursor, Windsurf, VS Code o Antigravity IDE para el ejercicio.

Debe entregar informe del ejercicio y código en GitHub y prompts utilizados."

El documento original es `SpringBoot AI.pdf`, ubicado en la raíz del proyecto.

Lee el documento original y utiliza sus requisitos como criterio de aceptación.

## 2. OBJETIVO

Preparar tres entregables finales:

**ENTREGABLE 1 — INFORME ACADÉMICO**

Un informe completo y profesional, en Word (.docx) y PDF (.pdf).

**ENTREGABLE 2 — CÓDIGO EN GITHUB**

El proyecto completo, correctamente organizado, documentado y verificado, disponible en:

https://github.com/Sbedoyar/ProyectoBanco

**ENTREGABLE 3 — PROMPTS UTILIZADOS**

Un documento organizado con los prompts reales empleados durante el desarrollo, disponible en Word, PDF y Markdown.

No basta con recomendar cómo hacerlo. Debes crear y actualizar los archivos correspondientes.

## 3. REVISIÓN INICIAL

Antes de comenzar:

- Revisa la estructura completa del proyecto.
- Analiza el estado actual del backend.
- Analiza el frontend React con sus módulos independientes.
- Revisa PostgreSQL y el modelo relacional.
- Revisa los scripts de ejecución y pruebas.

- Revisa README.md.
- Revisa docs/informe.md.
- Revisa docs/prompts.md.
- Revisa las evidencias disponibles en specs.
- Revisa las pruebas existentes.
- Revisa el estado del repositorio Git.

Utiliza los archivos actuales como fuente principal de información.

No inventes funcionalidades, estadísticas, resultados de pruebas ni detalles técnicos que no puedas verificar.

## 4. INFORME ACADÉMICO DEFINITIVO

Quiero un informe con calidad de presentación universitaria, bien redactado, técnicamente correcto y visualmente profesional.

El documento debe contener:

1. Portada académica.
2. Tabla de contenido.
3. Introducción.
4. Planteamiento del ejercicio.
5. Objetivo general.
6. Objetivos específicos.
7. Alcance del proyecto.
8. Requerimientos funcionales y no funcionales.
9. Tecnologías utilizadas.
10. Arquitectura general.
11. Explicación del monolito modular.
12. Estructura de carpetas del backend y frontend.
13. Modelo de base de datos.
14. Explicación de las entidades y sus relaciones.
15. Funcionamiento del registro e inicio de sesión.
16. Funcionamiento de las cuentas bancarias.
17. Proceso completo de transferencias.
18. Validaciones y reglas de negocio.
19. Integridad y atomicidad de las operaciones.
20. Histórico de movimientos.
21. Comprobantes y generación del QR.
22. Seguridad implementada.
23. Pruebas realizadas.
24. Evidencias de funcionamiento.
25. Instrucciones de instalación y ejecución.
26. Resultados obtenidos.
27. Limitaciones.
28. Conclusiones.
29. Referencia al repositorio GitHub.
30. Referencia al documento de prompts.

### Diseño del informe

Quiero un documento formal y bien presentado.

Utiliza:

- Portada elegante y académica.
- Tipografía profesional.

- Encabezados jerarquizados.
- Numeración de páginas.
- Tabla de contenido automática cuando sea posible.
- Tablas correctamente formateadas.
- Diagramas de arquitectura.
- Diagrama relacional de PostgreSQL.
- Diagramas de flujo cuando sean útiles.
- Capturas reales del frontend.
- Capturas del comprobante QR.
- Evidencias de pruebas y persistencia.

No quiero un documento lleno de texto repetitivo ni explicaciones artificialmente extensas.

El informe debe ser suficientemente detallado para demostrar el trabajo realizado y permitir su sustentación.

### Datos de portada

Utiliza únicamente los datos académicos que puedas confirmar.

Si falta el nombre del docente, grupo o integrantes, coloca campos pendientes claramente identificados, sin inventarlos.

### Archivos obligatorios

Genera:

- `docs/informe.md`
- `docs/Informe_ProyectoBanco.docx`
- `docs/Informe_ProyectoBanco.pdf`

Las versiones Word y PDF deben contener el mismo contenido académico y mantener una presentación coherente.

Verifica la paginación, las tablas, los diagramas y las imágenes antes de considerar terminados los documentos.

## 5. DOCUMENTO DE PROMPTS UTILIZADOS

El profesor exige presentar los prompts utilizados durante el desarrollo del ejercicio.

Revisa el historial existente y `docs/prompts.md`.

Organiza los prompts reales por fases:

1. Análisis y planeación.
2. Creación del backend.
3. Configuración de PostgreSQL.
4. Registro y autenticación.
5. Gestión de cuentas.
6. Transferencias.
7. Histórico de movimientos.
8. Generación de QR.
9. Pruebas y validaciones.
10. Refactorización a monolito modular.
11. Rediseño visual con frontend-design.
12. Organización del frontend por módulos.

13. Documentación.
14. Verificación final.
15. Preparación de la entrega.

Cada prompt debe incluir, cuando exista evidencia:

- Etapa del desarrollo.
- Objetivo.
- Prompt original completo.
- Herramienta utilizada.
- Resultado obtenido.

No inventes prompts históricos ni los presentes como transcripciones literales si solamente tienes un resumen.

Cuando un prompt original no esté disponible, identifica esa limitación.

Incluye el prompt actual como última etapa.

Genera:

- `docs/prompts.md`
- `docs/Prompts_Utilizados.docx`
- `docs/Prompts_Utilizados.pdf`

El documento debe ser limpio, organizado y legible.

## 6. EVIDENCIAS DEL PROYECTO

Utiliza las capturas reales disponibles en el repositorio y genera nuevas cuando resulte necesario.

El informe debe mostrar evidencias de:

- Inicio de sesión.
- Registro.
- Dashboard.
- Saldo de cuenta.
- Formulario de transferencia.
- Confirmación de transferencia.
- Comprobante QR.
- Histórico de movimientos.
- Vista de comprobantes.
- Interfaz responsive.
- Base de datos PostgreSQL.
- Pruebas ejecutadas.

Existe una transferencia manual comprobada entre las cuentas terminadas en 0003 y 0002 por $50.000 COP, donde el origen terminó con $950.000 y el destino con $1.050.000.

Utiliza esta evidencia solamente si existen capturas o registros accesibles que permitan documentarla correctamente. No inventes evidencias.

No incluyas contraseñas, secretos ni datos privados en capturas o documentos.

## 7. VERIFICACIÓN FINAL

Ejecuta los comandos de prueba existentes, incluyendo:

`./scripts/check-all.ps1`

Revisa qué comprueba realmente cada script.

Valida:

- Compilación backend.
- Pruebas Java.
- Integración PostgreSQL.
- Build React.
- Lint.
- Formato.
- Pruebas frontend.
- Pruebas Playwright.
- Navegación por módulos.
- Transferencias.
- Actualización de saldos.
- Histórico.
- QR.
- Persistencia.
- Seguridad.
- Idempotencia.
- Rollback.
- Concurrencia.

No afirmes que se aprobaron pruebas que no hayas ejecutado o cuyos resultados no estén disponibles.

Registra los números reales de pruebas aprobadas, fallidas y omitidas.

Si encuentras errores importantes que impiden la entrega, realiza correcciones mínimas y controladas, sin cambiar innecesariamente la arquitectura.

## 8. README FINAL DE GITHUB

Actualiza `README.md` para que funcione como presentación principal del repositorio.

Debe incluir:

- Nombre y descripción.
- Características principales.
- Tecnologías.
- Arquitectura general.
- Estructura del proyecto.
- Instrucciones para ejecutar PostgreSQL, Spring Boot y React.
- Ejecución de pruebas.
- Capturas del sistema.
- Explicación del histórico.
- Explicación del QR.
- Advertencia de simulación académica.
- Enlace al informe PDF.
- Enlace al informe Word.
- Enlace al documento de prompts.
- Enlace a la guía de sustentación.

Asegúrate de que todos los enlaces relativos funcionen correctamente en GitHub.

No agregues insignias de pruebas o métricas que no estén verificadas.

## 9. GUÍA DE SUSTENTACIÓN

Genera `docs/guia-sustentacion.md`.

Esta guía debe ayudarme a estudiar el proyecto y explicarlo al profesor.

Incluye explicaciones sencillas pero técnicamente correctas de:

- Arquitectura Full Stack.
- React y Spring Boot.
- API REST.
- PostgreSQL.
- Monolito modular.
- Controladores, servicios y repositorios.
- Entidades y DTO.
- Relaciones entre tablas.
- Flujo de una transferencia.
- @Transactional.
- BigDecimal.
- Bloqueos de cuentas.
- Rollback.
- Idempotencia.
- Histórico.
- QR.
- Seguridad.
- Pruebas unitarias, integración y E2E.

Incluye preguntas probables del profesor y sus respuestas.

Prepara también un guion de demostración del sistema, incluyendo el uso del frontend y consultas SQL de solo lectura en pgAdmin.

## 10. REVISIÓN DE GITHUB Y SEGURIDAD

El proyecto está publicado en:

https://github.com/Sbedoyar/ProyectoBanco

Ramas:

- `developer`: desarrollo.
- `main`: versión estable y predeterminada.

Trabaja exclusivamente sobre `developer`.

No modifiques `main`.

Comprueba que el repositorio no incluya:

- `.local/`
- `database.json`
- Credenciales.
- Contraseñas.
- Bases de datos locales.
- Archivos `.env` privados.

- Llaves privadas.
- `node_modules`.
- `target`.
- `dist`.
- Archivos temporales.

Inspecciona también documentación, capturas, archivos JSON e historiales internos para detectar información sensible.

No hagas push, merge ni cambios remotos sin autorización expresa.

## 11. MATRIZ DE CUMPLIMIENTO

Genera una matriz final que relacione cada requisito del profesor con:

- Funcionalidad implementada.
- Archivos responsables.
- Prueba correspondiente.
- Evidencia.
- Estado verificado.

Distingue lo implementado de lo comprobado.

Incluye esta matriz en el informe y, si resulta útil, también en `docs/verificacion-entrega.md`.

## 12. RESULTADO FINAL

Debes entregar:

1. Informe académico Word.
2. Informe académico PDF.
3. Informe Markdown actualizado.
4. Documento de prompts Word.
5. Documento de prompts PDF.
6. Registro Markdown de prompts.
7. README actualizado.
8. Guía de sustentación.
9. Matriz de cumplimiento.
10. Evidencias organizadas.
11. Resultados finales de pruebas.
12. Estado de Git y resumen de cambios.

Antes de terminar, comprueba que todos los archivos existen, que los PDF pueden abrirse y que su contenido es legible.

Si alguna herramienta necesaria para generar Word o PDF no está disponible, utiliza una alternativa compatible, preferiblemente sin instalar dependencias innecesarias. Si no es posible generar un archivo, informa la limitación sin afirmar que fue creado.

## 13. RESTRICCIONES

- No reconstruyas el proyecto.
- No agregues nuevas funcionalidades bancarias.
- No reemplaces la arquitectura.
- No elimines pruebas.
- No cambies contratos HTTP.
- No modifiques las reglas monetarias.
- No inventes resultados.

- No inventes prompts.
- No publiques secretos.
- No hagas push automáticamente.
- No hagas merge a main.

## INSTRUCCIÓN FINAL

Quiero que realices el trabajo completo y dejes ProyectoBanco preparado para su entrega académica del 30 de octubre de 2026.

Prioriza la calidad del informe, la exactitud de la documentación, el cumplimiento del profesor y la reproducibilidad de los resultados.

No te limites a presentar un plan: ejecuta las tareas, genera los documentos, realiza las verificaciones posibles y entrega un resumen final con los archivos creados, resultados de pruebas, pendientes reales y un mensaje de commit recomendado para `developer`.

**No consideres terminada la tarea hasta haber verificado los entregables generados.**